

AI 编程工作流 · 新项目速查

从零建项目 · 接入 Trellis · 用 Claude Code 走完 0→1 闭环 · 后续迭代模板 · 客户专属教学资料

阶段一：在 PowerShell 里建项目（只做一次）

```
# 1. 选个干净位置建新文件夹（D 盘也行）
cd D:\projects          # 没有就先 mkdir D:\projects
mkdir 新项目名
cd 新项目名

# 2. 初始化 git
git init

# 3. 接入 Trellis（Claude + Codex 都接的话用这条）
trellis init -u kelai --claude --codex --skip-existing

# 4. 启动 Claude
claude
```

三个最容易踩的坑

- `trellis init` 只跑这一次，第二天回来直接 `claude` 即可
- `git init` 必须在新项目目录里跑，不要在 `C:\Users\JyzZZ` 跑（否则把家目录变成仓库）
- `claude` 启动后看顶部路径，必须是新项目目录，不然 `/trellis:continue` 无效

阶段二：在 Claude 里立规范（bootstrap）

进 Claude 后第一句：

```
/trellis:continue
这是全新空项目。请用 get_context.py 和 task.py list 确认当前是不是 00-bootstrap-guidelines。
如果是，先说明它的作用，等我确认后归档。
```

它会问你**技术栈**。如果选项里没有合适的，选 `I'll describe it`，按这个模板回答：

Backend: [写你的后端: Node/Express, Python/FastAPI, 或"没有后端"]
Frontend: [写你的前端: React+TS, 纯 HTML/JS, 或"没有前端"]
数据库: [Postgres / SQLite / localStorage / 无]
测试: [是否要 Jest/pytest, 或"手工测试"]
目录: [单文件夹 / monorepo / 标准脚手架]
依赖偏好: [尽量少装 npm 包 / 用 xxx 框架]
Out of scope: [明确不做事, 比如登录、多人协作、Docker]

它会用你的答案去填 `.trellis/spec/backend/*.md` 和 `.trellis/spec/frontend/*.md`。

阶段三：建第一个真正的功能任务

bootstrap 归档后, 发:

为这个项目创建一个 `planning` 任务。
需求: [一句话写清你要什么]
范围: [列 3-5 个最小功能点]
验收标准 (必须可手工测试): [列 3-5 条具体能点击/能看到的现象]
Out of scope: [明确这次不做什么]
请加载 `trellis-brainstorm` 把 `prd.md` 补完整。先不要 `task.py start`, 也不要写代码。

关键技巧：第一版永远做"最小可用"。 比如 Todo 第一版只做"加 / 完成 / 删 / 持久化", 不要一上来塞登录 + 筛选 + 编辑 + 统计。增强功能放第二轮第三轮做。

阶段四：标准 6 步闭环（每个任务都这样跑）

步骤	你回 Claude 的话	它会做什么
1	PRD 看完 OK → "锁定 PRD，按 workflow 自动填 JSONL 并 task.py start"	把任务从 planning 翻到 in_progress
2	"开始实现"	派发 trellis-implement sub-agent，自动注入 spec + PRD，写代码
3	"dispatch trellis-check"	另一个 sub-agent 对照 spec 自检，自动修小问题
4	(自己开浏览器/终端手测一遍)	—— 这一步 AI 替代不了，必须人测 ——
5	"手测全过，请给 commit 建议"，确认清单后回 "确认提交"	跑 git add + git commit
6	<code>/trellis:finish-work</code>	归档任务 + 写 journal + 提交收尾 commit

阶段五：第二天回来续工

```
cd D:\projects\新项目名
claude
```

进 Claude 第一句**永远**是：

```
/trellis:continue
继续这个项目。请先读 get_context.py、task.py list、最新 journal。
然后告诉我：上次做到哪、当前完成度、建议下一步。先不要写代码。
```

阶段六：加功能 / 修 bug 的模板

每开一个新需求或新 bug，都是一个全新的小闭环。不要在同一个任务里堆功能。

加功能模板

```
/trellis:continue
开新任务: [一句话需求]
范围: [3-5 个功能点]
验收: [3-5 条手测标准]
Out of scope: [本轮不做什么]
请按 trellis-brainstorm 拆 PRD, 先不写代码。
```

修 bug 模板

```
/trellis:continue
有一个 bug, 请先确认当前任务和最新 journal, 再排查:
【现象】xxx
【复现】1. 第一步 2. 第二步 3. 第三步
【预期】xxx
【实际】xxx
修完告诉我根因、改了哪些文件、怎么验证。
如果反复修不对, 加载 trellis-break-loop 复盘。
```

速查表

时机	在 PowerShell	在 Claude
第一天建项目	<code>mkdir</code> + <code>git init</code> + <code>trellis init</code> <code>--claude --codex</code> + <code>claude</code>	<code>/trellis:continue</code> → 描述技术栈 → 建 planning 任务
每天开工	<code>cd</code> 到项目 + <code>claude</code>	<code>/trellis:continue</code> 让它先汇报上次进度
实现完成	(不用动)	<code>dispatch trellis-check</code> → 手测 → commit → <code>/trellis:finish-work</code>
新需求	(不用动)	<code>/trellis:continue</code> + 新需求模板

三条铁律

1. 不要跳过 PRD 直接写代码 —— PRD 是续工时唯一的锚点
2. 不要跳过手工测试就 commit —— `trellis-check` 不会真打开浏览器
3. 不要忘记 `/trellis:finish-work` —— 不归档下次 AI 看不到证据链

常见报错速查

报错	原因	解决
<code>&&</code> 不是有效语句分隔符	Windows PowerShell 5.1 不支持 <code>&&</code>	用 <code>;</code> 替代，或一行行执行；或换 Git Bash / PowerShell 7
<code>Unknown command: /trellis:continue</code>	Claude 启动时不在项目目录里	退出 Claude, <code>cd</code> 到项目, 再 <code>claude</code>
<code>permission denied</code> 装 trellis	macOS npm 全局目录权限	命令前加 <code>sudo</code>
<code>git has no identity configured</code>	git 没设 user.name / user.email	在 Claude 里输 <code>!git config user.name "你的名" && git config user.email "你@邮箱"</code>
PowerShell 显示中文乱码	默认 GBK 解码 UTF-8 文件	先执行 <code>chcp 65001</code> ；或用 VS Code 打开